

程序设计大作业报告

本项目是一款基于 Qt 框架的塔防游戏《保卫萝卜》，采用 C++ 编写，利用 QGraphicsView 实现 2D 场景渲染，支持关卡加载、塔防建造、波次怪物、炮塔升级与拆除、存档解锁等核心玩法。以下从整体架构、模块划分、类设计、游戏流程和关键机制等维度进行详细说明。

一、整体架构

游戏采用 MVC 变体架构，MainWindow 作为主控制器和视图容器，QGraphicsScene 承载所有游戏元素，QGraphicsItem 派生类充当模型和视图的双重角色。主要分层如下：

UI 层：顶部工具栏（金币、波数、倍速、暂停、返回）、菜单页、选关页、倒计时、结束弹窗。

逻辑层：怪物生成、波次管理、子弹飞行、碰撞检测、炮塔攻击、升级与拆除、游戏状态管理。

数据层：地图文件解析(.txt)、炮塔配置文件(towers.txt)、解锁进度持久化(unlocked.txt)。

资源层：图片、GIF 动画、样式表，以资源文件 (.qrc) 方式集成。

二、模块划分

1. 主控模块（MainWindow）

继承自 QMainWindow，负责整体界面布局、事件路由、游戏循环驱动。

管理多个页面（菜单、选关、游戏场景）的显示与切换。

持有所有游戏对象容器（怪物列表、地面格子、炮塔、子弹等）并协调其生命周期。

处理鼠标点击事件（通过事件过滤器），实现炮塔选择、放置、升级/删除菜单。

2. 游戏元素模块（QGraphicsItem 派生类）

ground：地面格子，标识是否可放置炮塔，存储格子类型（可建造/不可建造）。

Carrot：萝卜（基地），具有血量（hp），被怪物攻击会掉血，血量为 0 则游戏失败。

BaseMob / soldier / dog / tank：怪物基类与具体类型，具有生命值、移动速度、路径检查点、攻击萝卜行为。

Shooter：炮塔，具有等级、攻击范围、攻击间隔、总造价，可升级（最多 3 级）和拆除（返还 80% 金币）。

Shell：子弹，由炮塔发射，沿直线飞向目标怪物，命中后造成伤害并消失。

3. 路径与波次模块（数据结构）

Checkpoint：路径检查点，包含像素坐标和等待时间（用于怪物滞留）。

SpawnPoint：怪物侵入点，包含出生位置和检查点列表。

MobSpawnInfo：单只怪物的生成信息，包括侵入点 ID、类型、血量。

波次数据存储为 QList<QList<MobSpawnInfo>>，每个波次包含一组怪物生成信息。

4. 配置与持久化模块

地图解析：从 mapX.txt 读取网格、萝卜坐标、侵入点、检查点、波次数据，以及金币奖励（不同类型怪物击杀金币）。

炮塔配置：从 towers.txt 读取可用炮塔类型（当前仅一种）及其造价。

存档系统：通过 `unlocked.txt` 保存已解锁的最大关卡数（通关时萝卜血量 ≥ 8 即解锁下一关）。

5. 辅助模块

日志系统：`logInit()` / `logMsg()` 将调试信息输出到 `log.txt` 和控制台。

坐标转换：`gridToPixelX/Y` 将逻辑网格坐标映射到像素坐标（每个格子 80×80 像素）。

三、主要类设计

1. MainWindow 类

职责：游戏主窗口，控制整体流程与交互。

关键成员：

UI 组件：`topBar`, `goldText`, `waveText`, `countdownLabel`, `pauseBtn`, `speedBtn`, `menuBtn`。

页面容器：`menuPage`, `levelSelectPage`。

图形框架：`QGraphicsScene *scene`, `QGraphicsView *view`。

游戏对象容器：`QList<BaseMob*> *mobList`, `QList<ground*> groundList`, `QList<QGraphicsItem*> towerBarItems`, `QList<QGraphicsItem*> shooterCircles`。

核心数据：`carrot`, `spawnPoints`, `waves`, `totalWaves`, `currentWaveIndex`, `spawnIndex`。

状态标志：`gameStarted`, `gameOver`, `paused`, `doubleSpeed`, `shooterMenuMode`。

计时器：`timer`（驱动场景刷新），`spawnTimer`（怪物生成间隔），`waveIntervalTimer`（波次间隔）。

存档：`unlockedLevelCount`。

关键方法：

`loadLevel(int idx)`：加载指定关卡地图，解析文件，构建场景，启动倒计时。

`resetAndStart()`：重新加载当前关卡。

`clearGameObjects()` / `cleanupGame()`：清理所有游戏对象，恢复初始状态。

`enterShooterMenu(Shooter*)` / `exitShooterMenu()`：显示/隐藏炮塔升级/删除菜单。

`spawnNextMonster()`：按波次生成下一只怪物。

`startNextWave()`：开始下一波。

`togglePause()` / `doPause()`：暂停/恢复游戏。

`eventFilter`：拦截场景鼠标事件，处理炮塔选择、放置、升级/删除交互。

`updateGoldDisplay()`, `updateWaveDisplay()`, `updateTowerBarAppearance()`, `updateUpgradeHints()`：刷新界面状态。

2. ground 类

职责：表示地图上的一个网格格点，控制是否可以放置炮塔。

成员：

`int type`：1 表示可建造，2 表示不可建造（路径或障碍）。

`bool shooterPlaced`：是否已被炮塔占用。

方法：

`setShooterPlaced(bool)` / `clearShooterPlaced()` / `hasShooterPlaced()`。

3. Carrot 类

职责：代表萝卜（基地），拥有血量，可被怪物攻击。

成员：

`int hp`：当前血量，初始 10。

`bool isDead()`：血量 ≤ 0 时返回 `true`。

交互：

怪物抵达终点后会调用 `carrot->hp -= damage`，并触发游戏结束检测。

4. BaseMob 及其派生类

基类 BaseMob 继承 QGraphicsPixmapItem，抽象怪物通用行为。

成员：

`int hp, maxHp`: 当前血量与最大血量。
`int mobType`: 类型（1=士兵，2=狗，3=坦克）。
`QList<Checkpoint> checkpoints`: 路径点列表。
`int cplIndex`: 当前目标路径点索引。
`qreal speed`: 移动速度（像素/帧）。
`bool reachedEnd`: 是否到达终点。
`static Carrot *carrot`: 全局萝卜指针，用于攻击。

方法：

`advance(int phase)`: 实现移动逻辑，沿路径点前进，到达终点后攻击萝卜。
`takeDamage(int dmg)`: 扣血，若血量归零则死亡。

派生类：

`soldier`: 基础兵种，速度中等，血量中等。
`dog`: 速度较快，血量较低。
`tank`: 速度慢，血量高，嘲讽炮台。

5. Shooter 类

职责：炮塔，自动攻击进入射程的怪物。

成员：

`int level`: 1~3。
`int attackCooldown`: 攻击间隔（毫秒）。
`double rangePx`: 攻击范围半径（像素）。
`QList<int> upgradeCosts`: 各等级升级所需金币。
`int totalCost`: 累计投入金币（用于拆除返还）。

方法：

`upgrade()`: 提升等级，增加攻击力和范围，返回是否成功。
`getUpgradeCost()`: 返回下一级升级费用。
`getTotalCost()`: 返回总造价。
`advance(int phase)`: 每帧更新，寻找最近怪物，若冷却结束则发射子弹。

6. Shell 类

职责：子弹，由炮塔发射，沿直线飞向目标。

成员：

`QPointF targetPos`: 目标位置（怪物当前坐标）。
`BaseMob *target`: 目标怪物指针。
`qreal speed`: 飞行速度。
`int damage`: 伤害值。

方法：

`advance(int phase)`: 移动，检测是否命中目标或超出范围，命中后造成伤害并销毁。

四、游戏流程

1. 启动与菜单

构造函数初始化 UI，加载解锁进度，默认显示菜单页。

点击“选择关卡”进入选关页，已解锁的关卡可点击，未解锁的加锁图标并禁用。

2. 关卡加载

解析地图文件 `mapX.txt`：

读取金币奖励（三种怪物击杀所得）。

读取 7×13 网格，创建 `ground` 对象。

读取萝卜坐标，创建 `Carrot`。

读取侵入点数量及每个侵入点的检查点，创建 `SpawnPoint`，并添加侵入点图标。

读取波次数量及每波怪物列表。

加载炮塔配置，在场景顶部生成炮塔选择栏。

隐藏选关页，显示倒计时动画（2.1 秒），结束后启动游戏循环。

3. 游戏循环

主计时器 `timer` 以固定间隔（正常 32ms，2x 倍速 16ms）触发，驱动 `scene->advance()`，更新所有可移动对象（怪物、子弹、炮塔）。

波次管理：`waveIntervalTimer` 控制波次间隔（正常 5s，2x 2.5s），结束后调用 `startNextWave()`，递增波次索引，生成该波怪物。

怪物生成：`spawnTimer` 控制单只怪物的生成间隔（正常 1s，2x 0.5s），每次生成时从当前波次的 `spawnIndex` 读取配置，创建对应类型怪物并添加至场景。

游戏结束检测：

失败：萝卜血量归零 → 显示失败弹窗。

胜利：最后一波所有怪物被消灭且萝卜存活 → 判定胜利；若萝卜血量 ≥ 8 ，则解锁下一关（若当前关卡是已解锁的最高关卡）。

4. 用户交互

炮塔选择：点击炮塔选择栏卡片，高亮该卡片，进入“放置模式”。

炮塔放置：在放置模式下点击可建造且未被占用的格子，消耗金币建造炮塔，退出放置模式。

炮塔升级/删除：点击已有炮塔，弹出菜单（攻击范围圈、升级图标、删除图标）。点击升级图标消耗金币升级，点击删除图标返还 80% 金币并移除炮塔。

暂停/恢复：点击暂停按钮，暂停所有计时器并缓存剩余时间；再次点击恢复。

倍速切换：点击 2x 按钮，切换全局计时器间隔、怪物生成间隔、波次间隔，并调整现有计时器剩余时间。

返回菜单：点击返回按钮，弹出确认对话框，确认后清理游戏并回到菜单页。

5. 存档与解锁

`unlockedLevelCount` 存储在 `unlocked.txt`，初始为 1。

每当通关且萝卜血量 ≥ 8 时，若当前关卡索引等于 `unlockedLevelCount - 1`（即最新解锁关），则递增并保存。

选关页面根据 `unlockedLevelCount` 动态更新按钮锁状态。

五、机制详解

1. 怪物路径与移动

每个侵入点拥有一个检查点列表（按顺序）。怪物出生后，依次前往各检查点，到达最后一个检查点后视为抵达终点。

移动逻辑：每帧向当前目标点移动 `speed` 像素，到达后检查是否有等待时间（`waitTime`），若有则停留相应时间，然后转向下一检查点。

终点判定：当怪物到达最后一个检查点且无等待时间时，标记 `reachedEnd = true`，并调用 `carrot->takeDamage()`。

2. 炮塔攻击机制

每个炮塔在 `advance` 中扫描场景中所有怪物，计算距离，选择最近的活体怪物作为目标。攻击冷却结束后，发射子弹（Shell），子弹以一定速度飞向目标当前位置，命中后造成固定伤害（随等级提升）。

子弹飞行过程中，若目标死亡或离开射程，子弹仍继续飞向最后位置，超出范围后自毁。

3. 升级与拆除系统

炮塔升级所需金币：level 1→2 需 100，2→3 需 200（由 `upgradeCosts` 数组定义）。

升级后增加攻击力、攻击范围，并重置冷却。

拆除返还总投入金币的 80%（`totalCost * 0.8`），`totalCost` 记录建造及历次升级消耗的总和。

4. 金币与奖励

初始金币 500，击杀怪物获得金币（类型不同，由地图文件第一行指定）。

建造炮塔消耗金币，升级消耗金币，拆除返还金币。

界面实时更新金币显示，并影响炮塔选择栏的亮度及升级提示的可用性。

5. 性能优化

场景使用 `NoIndex` 索引方法，避免频繁重建索引。

使用 `BoundingRectViewportUpdate` 减少重绘区域。

怪物死亡后立即从场景移除，释放内存。

6. 事件处理与焦点

由于 `QGraphicsScene` 无法直接接收鼠标事件，通过 `installEventFilter` 拦截 `GraphicsSceneMouseEvent` 事件，实现点击交互。

所有菜单页（`QWidget`）置于场景之上，通过显示/隐藏控制，避免与场景事件冲突。

六. 分工安排

吴梓恒：负责基类及其方法设计

刘墨涵：负责 `mainwindow` 类的方法设计

陈奕鑫：负责美术元素和地图设计

七. 总结与改进

本项目遵循面向对象设计原则，将界面、逻辑、数据分离，通过 `Qt` 的图形视图框架高效管理大量动态对象。各模块职责清晰，扩展性良好（可方便增加新怪物类型、炮塔种类、关卡）。游戏状态支持暂停、倍速、中途返回等常用功能，存档系统保证了游戏进度。代码中大量使用日志便于调试。但在架构上有些问题，`mainwindow` 类设计过大，导致不利于分清逻辑关系，在指针运用上仍需学习。